

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 3

Выполнил:

Филатов Арсений Сергеевич

Группа К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Цель работы

Освоить практическое тестирование собственного REST API с помощью Postman: собрать один сквозной сценарий, отражающий основной бизнес-процесс приложения, и реализовать проверки (тесты) во вкладке Tests у запросов коллекции, чтобы сценарий можно было многократно прогонять (в том числе через Collection Runner) с контролем кодов ответа и структуры JSON.

2. Объект тестирования

Тестируется REST API сервиса поиска работы (Job Search API), реализованного в рамках ЛР1: стек Bun + Elysia, аутентификация через выдачу access/refresh токенов после регистрации и входа.

3. Задачи

Импортировать в Postman коллекцию с последовательностью запросов, описывающей основной пользовательский процесс соискателя.

Настроить переменные коллекции для адреса сервера и передачи данных между шагами (токены, идентификаторы вакансии и резюме).

Для ключевых запросов добавить Pre-request Script (при необходимости) и Tests с использованием pm.test, pm.expect (Chai), сохранением значений через pm.collectionVariables.set.

Обеспечить повторяемость сценария (уникальный email при регистрации за счёт timestamp).

Проверить выполнение сценария в Collection Runner при запущенном API и подготовленной БД (миграции + seed).

4. Комплексный сценарий

Сценарий по смыслу аналогичен примеру из задания (регистрация → вход → список → фильтрация → просмотр сущности → действие по процессу), но для предметной области вакансий и откликов:

5. Реализация тестов в Postman

5.1. Переменные коллекции. В JSON заданы, в частности: baseUrl (по умолчанию http://localhost:3000), candidateEmail, candidatePassword, accessToken, refreshToken, industryIdForFilter, vacancyId, resumeId. Значения токенов и идентификаторов не хранятся вручную — они выставляются из ответов предыдущих шагов.

5.2. Pre-request Script (шаг 1). Перед регистрацией формируется уникальный email вида pm_candidate_<timestamp>@test.local, что исключает конфликт 409 при повторных прогонах без очистки БД.

5.3. Вкладка Tests. Для каждого запроса добавлены проверки, например:

код ответа (pm.response.to.have.status(...));

структура JSON (pm.expect(json).to.have.property(...), проверка вложенных полей, массивов);

бизнес-инварианты (роль candidate, совпадение vacancyId/resumeId в отклике, наличие отклика в списке на финальном шаге).

Используется стандартная для Postman связка pm.test + Chai-ассерты (pm.expect).

5.4. Авторизация. Запросы после логина для защищённых эндпоинтов используют тип Bearer Token с подстановкой {{accessToken}}.

5.5. Зависимость от данных. Шаги 4–6 и выбор vacancyId рассчитаны на непустые справочники и хотя бы одну опубликованную вакансию; в тестах заложены сообщения о необходимости выполнить prisma db seed, если данных нет.

6. Условия прогона и порядок действий

Запустить PostgreSQL и API

В Postman: Import → файл lab1-collection.json.

При необходимости изменить переменную baseUrl.

Открыть Collection Runner, выбрать коллекцию, запустить все запросы по порядку.

Убедиться, что все pm.test отмечены как пройденные в отчёте Runner.

7. Результаты

Подготовлена Postman Collection с 11 последовательными запросами.

Реализованы автоматические тесты во вкладке Tests (плюс Pre-request Script на регистрации).

Сценарий соответствует требованию «один комплексный сценарий основного процесса» для job-платформы: от учётной записи до отклика и проверки списка откликов.

Сценарий воспроизводим при работающем сервере и заполненной тестовыми данными БД.

9. Выводы

Использование Postman позволило оформить API не только как набор ручных вызовов, но как регрессионно прогоняемый сценарий с явными критериями успеха в коде ответа и в теле JSON. Переменные коллекции и скрипты устраняют ручное копирование токенов и идентификаторов и приближают проверку к интеграционному тесту «через HTTP».